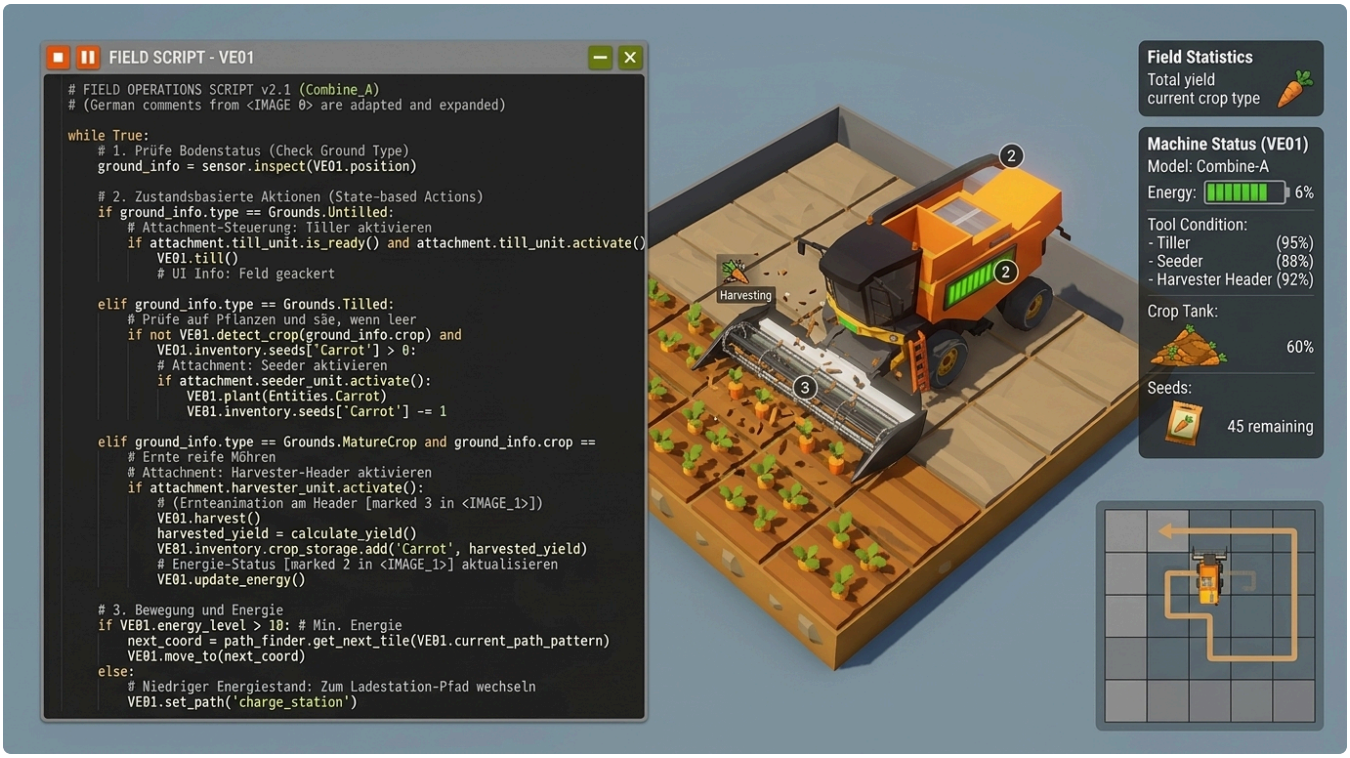
Plan Game: Ladang Otomatis dengan Kendaraan Pertanian

Dokumen perencanaan game mobile Android, terinspirasi dari The Farmer Was Replaced, dengan kendaraan pertanian darat sebagai pengganti drone. Dibuat dengan Google AI Studio.

1. Konsep Inti



Pemain menulis kode untuk mengontrol kendaraan pertanian (traktor, penabur, pemanen, pengangkut) di ladang. Tujuannya adalah mengotomatiskan pertanian dan mengoptimalkan hasil panen. Berbeda dengan drone, kendaraan darat punya batasan fisik: bahan bakar, jalur, kapasitas angkut, dan medan.

2. Perbedaan Utama dari Drone

- **Bahan bakar:** setiap gerakan mengurangi solar. Pemain harus mengatur rute dan mengisi ulang di pom bensin.
- **Medan:** tanah berlumpur memperlambat kendaraan, batu menghalangi jalan, dan jalan memberi bonus kecepatan.

- **Implement (alat tempel):** traktor harus memasang bajak, penabur, atau pemanen sebelum bekerja. Pergantian alat membutuhkan waktu.
- **Kapasitas:** hasil panen harus dibawa ke gudang dengan pengangkut, sehingga muncul masalah logistik.
- **Perawatan:** kendaraan bisa rusak dan perlu bengkel, sehingga kode harus memeriksa status kendaraan.

3. Core Loop

1. Tulis atau ubah kode.
2. Jalankan simulasi di ladang.
3. Amati hasil: panen, bahan bakar, waktu, dan error.
4. Optimalkan kode dan buka tantangan baru.

4. Sistem Kode

Bagian ini paling penting dan paling sulit. Jangan langsung memakai Python penuh di Android karena berat dan rawan keamanan. Dua pilihan yang lebih praktis:

- **Bahasa mini buatan sendiri:** lebih aman, ringan, dan mudah dijelaskan di tutorial.
- **Subset JavaScript** yang dijalankan di sandbox (misalnya Web Worker) dengan batas waktu eksekusi.

Contoh kode pemain:

```
for baris in range(5):
    traktor.maju(10)
    if traktor.bahan_bakar < 20:
        traktor.ke_pom_bensin()
    if tanaman_siap():
        traktor.panen()
    traktor.belok_kanan()
```

Fungsi dasar yang disarankan: `maju()`, `belok()`, `bajak()`, `tanam(bibit)`, `panen()`, `isi_bahan_bakar()`, `cek_tanah()`, `kirim_ke_gudang()`, dan `status()`.

5. Progres Permainan

- **Tahap awal:** satu traktor, satu lahan kecil, tugas membajak dan menanam.

- **Tahap menengah:** tambah alat, pengangkut, dan medan sulit.
- **Tahap akhir:** banyak kendaraan bekerja serentak tanpa tabrakan, serta efisiensi rute dan bahan bakar.
- **Pohon teknologi:** buka alat dan API baru dengan hasil panen.

6. Tech Stack

- **Prototipe:** Google AI Studio untuk versi web dengan React dan Canvas (PixiJS atau HTML Canvas biasa).
- **Editor kode:** CodeMirror, lebih ringan dan cocok untuk layar sentuh dibanding Monaco.
- **Gemini (opsional):** asisten yang memberi petunjuk saat pemain macet. Jadikan opsional agar game tetap jalan offline.
- **Ke Android:** ekspor kode, lalu bungkus dengan Capacitor untuk menghasilkan APK/AAB. Alternatifnya, jadikan PWA lalu bungkus dengan TWA.
- **Penyimpanan:** localStorage atau Capacitor Preferences.

7. Desain Mobile

- Layar dibagi dua: bagian atas untuk ladang, bagian bawah untuk editor kode.
- Tombol cepat untuk fungsi yang sering dipakai, agar pemain tidak terus mengetik.
- Dukungan mode landscape, lebih nyaman untuk ladang dan kode.
- Batas waktu eksekusi kode untuk menjaga baterai dan performa.

8. Rencana Pengembangan (perkiraan)

Minggu	Target
1	Desain API kode dan prototipe pergerakan kendaraan di grid
2	Bahan bakar, medan, dan alat tempel
3	Interpreter kode yang aman dan editor mobile
4	10 level pertama, sistem progres, dan penyimpanan data

Minggu	Target
5	Bungkus ke Android, uji di beberapa perangkat, perbaiki performa
6 dan seterusnya	Tambah level, polishing, dan persiapan rilis di Google Play

9. Risiko

- **Interpreter kode** adalah bagian tersulit. Mulai dengan subset kecil.
- **Google AI Studio** cocok untuk prototipe. Untuk produksi, kelola kode di repositori seperti GitHub.
- **Kebijakan Google Play:** pastikan aplikasi yang menjalankan kode memenuhi aturan, terutama jika ada fitur eksekusi kode.

10. Langkah Selanjutnya

- Tetapkan daftar API kode final.
- Buat struktur data level.
- Susun prompt awal untuk Google AI Studio agar prototipe langsung terbentuk.